

FCMP++

Luke "Kayaba" Parker

March 10, 2025

Abstract

FCMP++, shortened from FCMP+SA+L, short for Full-Chain Membership Proofs + Spend Authorization + Linkability, are an accomplishment of full-set privacy over the existing RingCT protocol used within Monero. This document serves as a specification and implementation reference.

1 Background

Monero is currently planning to deploy the Seraphis upgrade, most notable for its definition as a composition which distinguishes membership from spend authorization. With that, we are allowed much more efficient membership proofs (enabling full-set privacy). RingCT, which fails to so distinguish, was considered infeasible for full-set privacy for that reason and due to how its linking tags are defined.

With Seraphis, outputs are defined as $k_0 \cdot G_0 + k_1 \cdot G_1 + k_2 \cdot G_2$ with the linking tag defined as $(k_2/k_1) \cdot J$. This allows k_0 to be re-randomized without malleating the linking tag.

With a new, incompatible definition of linking tags, it forces creation of a new anonymity set (with explicit migration) to prevent double-spending of already spent outputs (which would now be given a new linking tag if 'imported' by simply adding a $1 \cdot G_2$ term).

With RingCT, outputs are defined as $O = x \cdot G$ with the linking tag defined as $x \cdot \text{HashToPoint}(O)$. To re-randomize x would be to malleate the linking tag.

2 New Output Key Definition

Since we cannot re-define the linking tags in an amenable fashion, we re-define the output key. We introduce a new generator, T , re-define output keys as $O = x \cdot G + y \cdot T$ (where $y = 0$ for all existing outputs), and preserve the linking tag definition of $x \cdot \text{HashToPoint}(O)$. This causes all existing outputs to maintain their linking tags while giving us a term to re-randomize.

3 Proof Separation

Seraphis defines four distinct proofs.

1. Membership Proof.

The tuple of the commitment being spent by this input and the output key associated with it is a member of the set of outputs within this blockchain.

2. "Ownership and Unspentness" (Spend Authorization and Linkability).

The transaction created is authorized by the possessor of the private keys for the output key. The commitment spent has not been prior spent.

3. Balance Proof.

This transaction does not create outputs worth more than the inputs spent.

4. Range Proof.

The outputs do not overflow the order of the elliptic curve in order to cheat the balance proof.

We appreciate and maintain these four definitions. We focus on the first two, currently jointly satisfied by CLSAG. We claim the following relationship currently satisfied.

$$\left\{ \begin{array}{l} G, H \in \mathbb{G}, \tilde{C}, L \in \mathbb{G}, S \in (\mathbb{G}, \mathbb{G}, \mathbb{G})^n; \\ O, I, C \in \mathbb{G}, x, c_g, c_h, r_c \in \mathbb{F} \mid \\ (O, I, C) \in S, \\ O = x \cdot G, L = x \cdot I, \\ \tilde{C} = (c_g + r_c) \cdot G + c_h \cdot H \end{array} \right\}$$

This performs set membership, proves possession of the private keys (so if the transcript is bound to the spend, performs spend authorization), and proves the linking tag is of the expected form (providing linkability).

We split this into two independent statements, augmented as necessary for composition into a statement functionally equivalent to the original.

3.1 New Membership Proof

OutputSet is defined as a set of (O, I, C) tuples (where O is the output key, $I = \text{HashToPoint}(O)$, and C the amount commitment).

For an output tuple (O, I, C) , the prover samples scalars r_o, r_i, r_j, r_c . They then prove knowledge of those scalars and:

$$\tilde{O} = O + r_o \cdot T$$

$$\tilde{I} = I + r_i \cdot U$$

$$R = r_i \cdot V + r_j \cdot T$$

$$\tilde{C} = C + r_c \cdot G$$

$$(O, I, C) \in \text{OutputSet}$$

while only publishing $\tilde{O}, \tilde{I}, \tilde{C}, R$ (the 'input tuple').

This does not require knowledge of x, y (nor the opening of the Pedersen commitment C) and accordingly can be produced by any party trusted with sender privacy. Exactly how this membership proof occurs is left as a black box to the rest of these designs, yet further specified later.

The typed statement is as follows.

$$\left\{ \begin{array}{l} G, T, U, V \in \mathbb{G}, \tilde{O}, \tilde{I}, R, \tilde{C} \in \mathbb{G}, \text{OutputSet} \in (\mathbb{G}, \mathbb{G}, \mathbb{G})^n; \\ O, I, C \in \mathbb{G}, r_o, r_i, r_j, r_c \in \mathbb{F} \mid \\ (O, I, C) \in \text{OutputSet}, \\ \tilde{O} = O + r_o \cdot T, \tilde{I} = I + r_i \cdot U, R = r_i \cdot V + r_j \cdot T, \tilde{C} = C + r_c \cdot G \end{array} \right\}$$

3.2 New Spend Authorization and Linkability Proof

For an input tuple $\tilde{O}, \tilde{I}, \tilde{C}, R$, and linking tag L , prove knowledge of x, y', r_i such that $\tilde{O} = x \cdot G + y' \cdot T, R = r_i \cdot V + r_j \cdot T, \tilde{I} = I + r_i \cdot U$, and $L = x \cdot I$ (while binding to the transaction hash). To be explicit, $y' = y + r_o$.

The typed statement is as follows.

$$\left\{ \begin{array}{l} G, T, U, V \in \mathbb{G}, \tilde{O}, \tilde{I}, R, L \in \mathbb{G}; \\ x, y', r_i, r_j \in \mathbb{F} \mid \\ \tilde{O} = x \cdot G + y' \cdot T, R = r_i \cdot V + r_j \cdot T, L = x \cdot \tilde{I} - (x * r_i) \cdot U \end{array} \right\}$$

We instantiate this literally with a Bulletproof+ (<https://eprint.iacr.org/2020/735>) composed with a Generalized Schnorr Protocol (<https://eprint.iacr.org/2009/050>).

We start by inlining the math for the Bulletproof+ Weighted-Inner Product proof (in its single round form with its $y = 1$, as that's all we need).

1. Prover samples $r_p, \alpha, \beta, \mu, \delta$ and publishes:

- $P = x \cdot G + r_i \cdot V + (x * r_i) \cdot U + r_p \cdot T$

This is the P from the statement of Bulletproof+'s weighted-inner-product proof for $\mathbf{g} = [G], \mathbf{h} = [V], g = U, h = T$ (in its own notation).

- $A = \alpha \cdot G + \beta \cdot V + ((\alpha * r_i) + (\beta * x)) \cdot U + \delta \cdot T$
- $B = (\alpha * \beta) \cdot U + \mu \cdot T$

2. Verifier sends challenges e .

3. Prover publishes:

- $s_\alpha = \alpha + e \cdot x$
- $s_\beta = \beta + e \cdot r_i$
- $s_\delta = \mu + \delta * e + r_p * (e * e)$

4. Verifier checks:

- $(e * e) \cdot P + e \cdot A + B = (s_\alpha * e) \cdot G + (s_\beta * e) \cdot V + (s_\alpha * s_\beta) \cdot U + s_\delta \cdot T$

We now have $P = x \cdot G + r_i \cdot V + (x * r_i) \cdot U + r_p \cdot T$ for some x, r_i, r_p . We set $P' = P - \tilde{O} - R$. For $P, \tilde{O}$, the only G term is for x . If the x s are consistent, P' will not have a G term. For P, R , the only V term is for r_i . If the r_i s are consistent, P' will not have a V term. For an honest prover, this leaves $P' = (x * r_i) \cdot U + (r_p - y' - r_j) \cdot T$.

We now compose this with a Generalized Schnorr Protocol to:

- Open P' over U, T , proving x and r_i were consistent.
- Prove the linking tag is well-formed.

1. Prover samples r_x, r_y, r_z, r_{r_p} and publishes:

- $R_O = r_x \cdot G + r_y \cdot T$
This is a commitment to the nonces we use when showing we know an opening of $\tilde{O}$.
- $R_P = r_z \cdot U + r_{r_p} \cdot T$
This is a commitment to the nonces we use when showing we know an opening of P' .
- $R_L = r_x \cdot \tilde{I} + r_z \cdot -U$
This is a commitment to the nonces we use when showing we know a consistent opening of L .

2. Verifier sends challenges c .

3. Prover publishes:

- $s_x = r_x + c \cdot x$
- $s_y = r_y + c \cdot y'$
- $s_z = r_z + c \cdot (x * r_i)$

This implicitly makes z , never explicitly defined, $x * r_i$.

- $s_{r_p} = r_{r_p} + c \cdot (r_p - y' - r_j)$
 $r_p - y' - r_j$ is the coefficient for the T term of P' .

4. Verifier checks:

- $R_O + c \cdot \tilde{O} == s_x \cdot G + s_y \cdot T$
This verifies the opening of $\tilde{O}$.
- $R_P + c \cdot P' == s_z \cdot U + s_{r_p} \cdot T$
This verifies the opening of P' over U, T .
- $R_L + c \cdot L == s_x \cdot \tilde{I} + s_z \cdot -U$
This verifies the opening of L as $x \cdot \tilde{I} + (x * r_i) \cdot -U$. Since $x \cdot \tilde{I}$ is $x \cdot I + (x * r_i) \cdot U$, this will remove the additional term (proved correct and consistent with the Bulletproof+), leaving $x \cdot I$ (the intended linking tag).

In the future, we should review specialized proofs. It would not be surprising if s_x can be replaced for just s_α , along with a few other similar tweaks.

4 Membership Proof

With the proofs separated, and spend authorization satisfied, we still need membership. We prove membership over the entire output set using an arithmetic circuit verifying a Merkle tree proof. Since Merkle tree proofs have $O(\log n)$ time complexity, a proof with $O(n)$ complexity still achieves a $O(\log n)$ solution to the problem statement.

This makes the requirement a sufficiently performant, zero-knowledge proof for the satisfaction of an arithmetic circuit, yet not explicitly a SNARK (which would mandate sublinear time complexity).

4.1 Arithmetic Circuit Proof

Bulletproofs are a trustless proof already used within Monero, which can be used to prove arithmetic circuit proofs are satisfied. We solely need a hash function for the Merkle tree, the most efficient for our use-case being a Pedersen hash.

A Pedersen hash is a collision-resistant hash where for each word hashed, an additional term is added to a Pedersen Vector Commitment. Finding a collision implies finding the discrete log relationship of the generators used.

Creating Pedersen hashes in-circuit would be suboptimal. Even with native arithmetic, by having the Bulletproof be proven over a curve whose scalar field is the field of the curve the hash is performed over, we'd have a cost of $\tilde{10}$ in-circuit multiplications per word hashed.

We instead use Generalized Bulletproofs, a modification to Bulletproofs recently proven secure: <https://github.com/cyphertack/generalized-bulletproofs>

Generalized Bulletproofs can be used to output Pedersen hashes at effectively no additional cost, although it cannot operate over its hashes (and we do need to operate over a series of hashes, due to utilizing a Merkle tree). We solve this by operating over output hashes with a new Generalized Bulletproof, this one on a curve whose scalar field is the prior curve's field. This continues for each layer of the Merkle tree, and is most efficiently realizable with a curve cycle (a curve whose scalar field is the field of a curve whose scalar field is the field for the original curve). With a curve cycle, one can alternate between the two curves (and two proofs) without needing additional curves/proofs per layer.

The ability to produce Pedersen hashes also enables forming challenges off partial transcripts of the witness. By defining a Pedersen hash of a set of variables within the circuit, we can hash that outside of the circuit with a traditional hash function to obtain a challenge usable for the rest of the circuit. This allows interactive proofs within the circuit (due to the Fiat-Shamir transform), and with it, more efficient gadgets.

For the concept of Generalized Bulletproofs, and opening a Merkle tree using Pedersen hashes with a pair of Bulletproofs over a curve cycle in such an efficient manner, please thank the authors of Curve Trees, <https://eprint.iacr.org/2022/756>. This proposal applies their work to RingCT and performs further optimizations via the usage of divisors, yet does not claim to be novel.

4.2 Towering Curve Cycle

Since we're performing full-chain membership proofs over Monero's RingCT, and Monero's RingCT uses Ed25519, we need a curve cycle involving Ed25519. Unfortunately, a curve cycle with Ed25519 is impossible for non-trivial curves. This is due to Ed25519 having a cofactor of 8. Instead of forming a curve cycle with Ed25519, we find a prime-order curve whose scalar field is Ed25519's field, and then form a curve cycle with that. This is referred to as a towering curve cycle, due to it being a curve cycle where one curve towers (stands over, not forming a cycle with) Ed25519.

tevador proposed Helios and Selene for an efficient towering curve cycle, documenting the deterministic method used to find them and their security. Please see <https://gist.github.com/tevador/4524c2092178df08996487d4e272b096>.

4.3 Gadgets

We define an arithmetic circuit with two parts to its statement.

- $a_l * a_r = a_o$

For vectors a_l, a_r, a_o , a_o is the Hadamard product of a_l, a_r .

- $W_l * a_l + W_r * a_r + W_o * a_o + W_v * V + c = 0$

W_l, W_r, W_o, W_v, c have an equal amount of columns. An instance of this formula, referred to as a constraint, exists per column. Within an instance, we refer to W_l, W_r, W_o, W_v, c as the column relevant (and not the overall structure).

W_l, W_r, W_o, W_v has as many rows as a_l, a_r, a_o . W_v additionally has a depth equivalent to the amount of Pedersen vector commitments. c is a single value.

$W_{l,r,o} * a_{l,r,o}$ is $\sum_{i=0}^{a_{l,r,o}.len()} W_{l,r,o,i} * a_{l,r,o,i}$ (where l, r, o refers to one of the relevant three).

$W_v * V$ is $\sum_{i=0}^{V.len()} \sum_{j=0}^{a_{l,r,o}.len()} W_{v,i,j} * V_{i,j}$

Please note W_v, V is defined by Bulletproofs(+) to be the Pedersen commitments accessible within the circuit and associated weights. We define V as the Pedersen vector commitments, with the necessary sub-indexing added. While Generalized Bulletproofs still supports Pedersen commitments, we drop the functionality from consideration and don't bother notating it due to lack of use.

Please also note Bulletproofs(+) place the constants c on the right-hand side, not the left. This means converting from our linear combinations to one for Bulletproofs(+) require negating the constants c .

A gadget is a series of a_l, a_r (and respective a_o) rows and columns in W_l, W_r, W_o, W_v, c . They are intended to be reusable snippets we can independently formally verify as correct/create security proofs for.

We define $a_l.push, a_r.push, V_i\emptyset$ as functions pushing a linear combination or value onto the respective vector, returning a constrainable reference. Once $a_l.push, a_r.push$ have been called, the push to a_o is implicit with a constrainable reference retrievable with $a_o.last()$.

4.3.1 Equality

$equality(a, b) \rightarrow$

$$1 \ a + -1 \ b = 0$$

By adding b to both sides, we get $a = b$.

4.3.2 Inverse

```

Inverse( $a$ ) :  $z \rightarrow$ 
  Equality( $a_l.\text{push}(a)$ ,  $a$ )
   $z = a_r.\text{push}(a^{-1})$ 
   $1a_o.\text{last}() + -1 = 0$ 
  return  $z$ 

```

For $a * z = 1$, we can rewrite this as $z = 1/a$ (which is the multiplicative inverse of a).

This also serves to prove $a \neq 0$ since if it was 0, it would produce a product of 0 (which is not 1).

4.3.3 Inequality

```

inequality( $a, b$ )  $\rightarrow$ 
   $c = 1 a + -1 b$ 
  Equality( $a_l.\text{push}(c)$ ,  $c$ )
   $a_r.\text{push}(c^{-1})$ 
   $1 a_o.\text{last}() + -1 = 0$ 

```

We constrain c to be $a - b$. Then we constrain c to having a multiplicative inverse. If $a = b$, c will be 0, and $a_o.\text{last}()$ must be 0 (since any number multiplied by 0 is 0).

4.3.4 Member of List

```

MemberOfList( $L, m$ )  $\rightarrow$ 
   $c = 1L_0 + -1m$ 
   $\forall 1 \leq i < L.\text{length}() - 1 :$ 
    Equality( $a_l.\text{push}(c)$ ,  $c$ )
     $n = 1L_i + -1m$ 
    Equality( $a_r.\text{push}(n)$ ,  $n$ )
     $c = 1a_o.\text{last}()$ 
   $c = 0$ 

```

This defines a carried multiplication of $L_i - m$, which will be 0 if and only if $L_i = m$. The multiplication of two values will be 0 if and only if one factor is 0 (since we're operating over a prime field). Accordingly, the carried multiplication will be 0 only if at least one $L_i = m$.

4.3.5 On Curve

```

OnCurvea,b( $x, y$ ) →
  Equality( $a_l.push(x), x$ )
  Equality( $a_r.push(x), x$ )
   $x^2 = a_o.last()$ 
  Equality( $a_l.push(x^2), x^2$ )
  Equality( $a_r.push(x), x$ )
   $x^3 = a_o.last()$ 
  Equality( $a_l.push(y), y$ )
  Equality( $a_r.push(y), y$ )
   $y^2 = a_o.last()$ 
  Equality( $1\ y^2, 1\ x^3 + a\ x + 1\ b$ )

```

This evaluates the curve formula of the embedded curve (written in the form $y^2 = x^3 + a * x + b$).

4.3.6 Incomplete Addition

```

IncompleteAddition( $x_0, y_0, x_1, y_1, x_2, y_2$ ) →
  Inequality( $x_0, x_1$ )
   $\delta = (y_1 - y_0) / (x_1 - x_0)$ 
   $\delta = a_l.push(\delta)$ 
   $r = a_r.push(x_1 - x_0)$ 
   $1\ x_1 + -1\ x_0 + -1\ r = 0$ 
   $1\ y_1 + -1\ y_0 + -1\ a_o.last() = 0$ 
   $\delta_1 = a_l.push(\delta)$ 
   $r = a_r.push(x_2 - x_0)$ 
  Equality( $\delta, \delta_1$ )
   $1\ x_2 + -1\ x_0 + -1\ r = 0$ 
   $-1\ y_2 + -1\ y_0 + -1\ r = 0$ 
  Equality( $a_l.push(\delta), \delta$ )
  Equality( $a_r.push(\delta), \delta$ )
  Equality( $1\ x_0 + 1\ x_1 + 1\ x_2, a_o.last()$ )

```

4.4 Interactive Gadgets

The rest of the gadgets we define are interactive. This bounds all of their arguments to being within Pedersen vector commitments, and has them called with an additional, per-gadget series of challenges chl . chl is derived from hashing all of the Pedersen vector commitments. Each gadget is presumed to be given an independent series of challenges, subscripted chl_i as needed.

4.4.1 Tuple Member of List

$\text{TupleMemberOfList}(L, m) \rightarrow$
 $\forall 0 \leq i < L.\text{length}() :$
 $\forall 0 \leq j < L_i.\text{length}() :$
 $L_i = \sum_{j=0}^{L_i.\text{length}()} \text{chl}_j * L_{i,j}$
 $m = \sum_{j=0}^{m.\text{length}()} \text{chl}_j * m_j$
 $\text{MemberOfList}(L, m)$

For this to have an attack in polylogarithmic time, it'd presumably need to be converted to the birthday problem. Since changing the tuple claimed to be a member (the left-hand side) changes the effective list (each member being a potential right-hand side), the two sides aren't independent and Wagner's algorithm isn't usable to find an efficient solution.

4.4.2 Discrete Log Proof

We use elliptic curve divisors, as posited by Liam Eagen (<https://eprint.iacr.org/2022/596>), to prove for the statement:

$$\{q, x, y; s_{0..256} \mid (x, y) = ((\sum_{i=0}^{255} 2^i * s_i) \bmod q) \cdot G\}$$

We do this by proving for the functionally equivalent statement:

$$\{q, x, y; s_{0..256} \mid (x, y) = \sum_{i=0}^{255} s_i \cdot (2^i \cdot G)\}$$

where G is of prime order q .

The scalar represented by s , s' , is equal to $\sum_{i=0}^{255} 2^i * s_i \bmod q$. Since we do not constrain s to being a valid bitstring, each s' has effectively infinite representations which can be used for s . We solely require s' not be malleable (for a given s , the prover should not be able to prove for multiple s').

```

DivisorChallengea,b( $d_y, d_{yx_{0..m}}, d_{x_{1..n}}, d_0, \delta, \text{chl}$ )  $\rightarrow$ 
 $p_{0_{n_0}} = (3 * \text{chl.x}^2 + a) / (2 * \text{chl.y})$ 
 $p_{0_{n_1}} = \sum_{j=0}^m (p_{0_{n_0}} * \text{chl.x}^{j+1}) * d_{yx_j} + (p_{0_{n_0}} * d_y)$ 
 $p_{0_{n_2}} = \text{chl.y} * d_{yx_0} + \sum_{j=1}^m ((j+1) * \text{chl.y} * \text{chl.x}^j) * d_{yx_j} + \sum_{i=1}^n ((i+1) * \text{chl.x}^i) * d_{x_i} + 1$ 
 $p_{0_n} = p_{0_{n_1}} + p_{0_{n_2}}$ 
 $p_{0_d} = \text{chl.y} * d_y + \sum_{i=0}^m (\text{chl.y} * \text{chl.x}^{(i+1)}) * d_{yx_i} + \sum_{i=1}^n \text{chl.x}^{(i+1)} * d_{x_i} + 1 * \text{chl.x} + 1 * d_0$ 
 $p_{1_n} = 2 * \text{chl.y}$ 
 $p_{1_d} = (-\delta * p_{1_n}) + 3 * \text{chl.x}^2 + a$ 
 $p_n = p_{0_n} * p_{1_n}$ 
 $p_d = p_{0_d} * p_{1_d}$ 
Equality( $a_l.\text{push}(p_d), p_d$ )
 $o = a_r.\text{push}(p_n * p_d^{-1})$ 
Equality( $a_o.\text{last}(), p_n$ )
return  $o$ 

```

d_y, d_{yx}, d_x, d_0 are the coefficients for the divisor (zero-indexed, such that d_{x_0} is the coefficient for x^1). The divisor is enforced to be non-zero by using 1 for the coefficient for x^1 . This is why the coefficients d_x start from 1 (omitting d_{x_0}).

While evaluating divisors (a polynomial), we consider the evaluation not as $a * x^2 + b * x + c$ (a generic polynomial used for this example), yet $x^2 * a + x * b + c$. The latter lines up with the constraint system's definition (since our x is public yet our coefficients are not).

a, b are the a, b from the curve equation $y^2 = x^3 + a * x + b$.

Please note p_{1_n}, p_{1_d} are scalars, not linear combinations, despite our extensive usage of linear combinations here.

```

DiscreteLogG(s0..256, x, y) →
OnCurvea,b(x, y)
dy, dyx0..m, dx1..n, d0 ← Prover (Vector Commitment)
chl0 =  $\mathcal{H}_{point}$ (chl0)
chl1 =  $\mathcal{H}_{point}$ (chl1)
chl2 = -(chl0 + chl1)
δ = (chl1.y - chl0.y) * (chl1.x - chl0.x)-1
μ = chl0.y - (δ * chl0.x)
f = Inverse(μ - (-y - (δ * x)))
Equality( $\sum_{i=0}^2$  DivisorChallengea,b(dy, dyx, dx, d0, δ, chli),  $\sum_{i=0}^{255} (\mu - (G_i.y - (\delta * G_i.x)))^{-1} * s_i + 1 f$ )

```

For a public generator G , both the prover and the verifier calculate $G_0 = 2^0 \cdot G, \dots, G_{255} = 2^{255} \cdot G$. The prover provides s , the presumed (yet not enforced) 256-bit decomposition of a scalar for the embedded elliptic curve, and the divisor interpolating $-(s \cdot G)$ with the relevant powers of G within Pedersen vector commitments.

We then perform a challenged evaluation of the divisor for the left-hand side of both the original equation and our equality statement. For the right-hand side, the powers of two of the generator are also so challenged, yet only included as selected by the vector s . The vector s may trigger multiple inclusions of one of the generators within our table. Since the individual term is written $1/(\mu - (G.y + (\delta * G.x)))$, a s_i of two causes $2/(\mu - (G.y + (\delta * G.x)))$ which is equivalent to $1/(\mu - (G.y + (\delta * G.x))) + 1/(\mu - (G.y + (\delta * G.x)))$. Since this is evaluated within a sum statement, this is equivalent to that specific G being interpolated multiple times, which is safe according to Eagen's preprint. Since reuse of s will cause a distinct generator's power of two with the same position to be interpolated the same amount of times, this does yield a consistent s' (with the assumption all generators share an order).

4.5 Circuit

We define two distinct layer gadgets, one for the first layer (which ensures the integrity of the tuple output), and one for all additional layers.

4.5.1 First Layer

FirstLayer($\tilde{O}, \tilde{I}, \tilde{C}, R$) $\rightarrow$

$o_x, o_y, r_{o_{0..255}}, r_{o_x}, r_{o_y} \leftarrow \text{Prover (Vector Commitment)}$
 $\text{OnCurve}_{a,b}(o_x, o_y)$
 $\text{DiscreteLog}_T(r_o, r_{o_x}, r_{o_y})$
 $\text{IncompleteAddition}(o_x, o_y, r_{o_x}, r_{o_y}, \tilde{O}.x, \tilde{O}.y)$

$i_x, i_y, r_{i_{0..255}}, r_{i_{u_x}}, r_{i_{u_y}}, r_{i_{v_x}}, r_{i_{v_y}} \leftarrow \text{Prover (Vector Commitment)}$
 $\text{OnCurve}_{a,b}(i_x, i_y)$
 $\text{DiscreteLog}_U(r_i, r_{i_{u_x}}, r_{i_{u_y}})$
 $\text{IncompleteAddition}(i_x, i_y, r_{i_{u_x}}, r_{i_{u_y}}, \tilde{I}.x, \tilde{I}.y)$

$r_{j_{0..255}}, r_{j_x}, r_{j_y} \leftarrow \text{Prover (Vector Commitment)}$
 $\text{DiscreteLog}_V(r_i, r_{i_{v_x}}, r_{i_{v_y}})$
 $\text{DiscreteLog}_T(r_j, r_{j_x}, r_{j_y})$
 $\text{IncompleteAddition}(r_{i_{v_x}}, r_{i_{v_y}}, r_{j_x}, r_{j_y}, R.x, R.y)$

$c_x, c_y, r_{c_{0..255}}, r_{c_x}, r_{c_y} \leftarrow \text{Prover (Vector Commitment)}$
 $\text{OnCurve}_{a,b}(c_x, c_y)$
 $\text{DiscreteLog}_G(r_c, r_{c_x}, r_{c_y})$
 $\text{IncompleteAddition}(c_x, c_y, r_{c_x}, r_{c_y}, \tilde{C}.x, \tilde{C}.y)$

$L \leftarrow \text{Prover (Vector Commitment)}$
 $\text{TupleMemberOfList}(L, (o_x, o_y, i_x, i_y, c_x, c_y))$

4.5.2 Additional Layer

AdditionalLayer($\tilde{H}$) $\rightarrow$

$h_x, h_y, r_{0..255}, r_x, r_y \leftarrow \text{Prover (Vector Commitment)}$
 $\text{DiscreteLog}_H(r, r_x, r_y)$
 $\text{OnCurve}_{a,b}(h_x, h_y)$
 $\text{IncompleteAddition}(h_x, h_y, r_x, r_y, \tilde{H}.x, \tilde{H}.y)$

$L \leftarrow \text{Prover (Vector Commitment)}$
 $\text{MemberOfList}(L, h_x)$

Please note H is the blinding generator for vector commitments on the previous curve (not the unblinded hash, which is h_x, h_y).

4.6 Tying It All Together

The first layer (performed within a Generalized Bulletproof over the Selene curve) proves knowledge of the existence of O, I, C points before being re-randomized by solely the intended blinding generators for each element, and that R is well-formed. It then proves the points are members of the list hashed into a vector commitment. It does not prove nothing else is hashed into the vector commitment (it doesn't constrain other members of the commitment are equal to zero) as that is unnecessary.

The prover and verifier take the vector commitment, a point on the Selene curve, and add B_{selene} (a public parameter sampled uniformly from random). The resulting point is passed as $\tilde{H}$ to the **AdditionalLayer** gadget within the Generalized Bulletproof over the Helios curve if another layer exists.

AdditionalLayer proves knowledge of a point h_x, h_y before re-randomizing by solely the intended blinding generator H . It then proves the x -coordinate is a member of the list hashed into a vector commitment. The prover and verifier take this vector commitment, add B_{curve} , where **curve** is the curve the vector commitment is on, and passes the result to the **AdditionalLayer** gadget within the Generalized Bulletproof over the reciprocal curve if another layer exists. If this vector commitment is for the root of the Merkle tree, the prover performs a discrete-log proof of knowledge for the difference of their vector commitment and the declared Merkle tree's root over the blinding generator for the vector commitment.

The verifier additionally checks the declared Merkle tree's root is acceptable (on-chain) and that the amount of layers executed is the amount of layers expected for the Merkle tree.

4.7 On Only Hashing x -coordinates Within AdditionalLayer

At each additional layer, only x -coordinates of the vector commitments are hashed for efficiency's sake. This would enable proving that either a branch $\tilde{H}$ is in-tree or its additive inverse, $-\tilde{H}$. We prevent that not by hashing $\tilde{H}$ yet $B_{\text{curve}} + \tilde{H}$. Attempting to prove for $-\tilde{H}$ attempts to prove $B_{\text{curve}} - \tilde{H}$ is in-tree, where $B_{\text{curve}} + \tilde{H}$ and $B_{\text{curve}} - \tilde{H}$ have distinct x -coordinates. While there would be an x -coordinate collision if $\tilde{H}$ had $2 \cdot B_{\text{curve}}$ subtracted, the Generalized Bulletproof proves knowledge of an opening of $\tilde{H}$ over a specific set of generators (preventing inclusion of B_{curve} terms).

Credit for the idea of B_{curve} goes to tevador.

5 Functionality

With the system described, it is important to be clear on the functionality in order to fairly evaluate it.

5.1 Full-Set Privacy

This is the reason for discussing this in the first place.

5.2 Hardware Wallets

Hardware wallets maintain their support due to not needing to perform the membership proof, solely the Generalized Schnorr Protocol after the fact. The Generalized Schnorr Protocol is smaller than the current CLSAG proofs, and is accordingly assumed to be well within the allowed memory footprint.

5.3 Multisignature Protocols

Generalized Schnorr Protocols effectively are Schnorr signatures, as the name implies. Accordingly, they benefit from all of the great work performed on Schnorr multisignatures, and can be expected to offer low-complexity, small-constant multisignature protocols.

5.4 Outgoing View Keys

Outgoing view keys, a new private key which enables calculating linking tags for scanned outputs without possession of the private key, are inherently enabled by this proposal. Since x, y are needed to spend, yet the linking tag is solely derived from x , x becomes the shareable outgoing view key. This does require y be unknown to whoever the outgoing view key is shared to, which is not the case for historical outputs (where $y = 0$).

If wallets publish addresses whose public spend key is not $s \cdot G$ yet $o \cdot G + y \cdot T$, where y is the effective private spend key (uniformly sampled), o , the outgoing view key, can be shared. This solely requires wallet software update their key handling internally, and can be done at any moment in time, by any subset of wallets, with no impact to privacy (on- or off-chain) nor network performance.

5.5 Forward Secrecy

We define forward secrecy as an adversary with a discrete log oracle being able to, for any output, find a consistent opening for the given input tuple $(\tilde{O}, \tilde{I}, \tilde{C}, R)$ and its Spend Authorization + Linkability proof.

Per the following Python script,

<https://gist.github.com/kayabaNerve/e09ba62d4a6165ced006b037f1068d95>

this is true. The script creates an output, decides all of the necessary nonces, and publishes its solutions. The script proceeds to create a random output (specifically, a random linking tag generator for an output), and from there performs extraction of the various witness values/nonces which would have been used. Since it is able to extract a solution for every variable, and rebuild identical commitments, it is able to forge a indistinguishable proof that output was the output spent. Since any output will have an indistinguishable forged proof, one cannot distinguish a legitimate proof.

An adversary with a discrete log oracle also cannot distinguish between an unspent non-forward-secret output and a forward-secret output. Such an adversary can only calculate what the linking tag would be if the output isn't forward secret, and wait to see if that appears (making it a spent non-forward-secret output).

5.6 Transaction Chaining

Since the membership proof does not require knowledge of x, y , the membership proof can be produced after the transaction itself is signed. This would let Alice and Bob form a 2-of-2 multisig, sign a transaction spending an output sent to it (per some mutually known blinding factors), then let Alice or Bob create that output, and once it's past the 10-block lock, publish the transaction spending it without the other party's participation.

We do have to ensure the message the Generalized Schnorr Protocol signs isn't binding to the membership proof however.

6 Integration

With the concepts of the proof established, and the functionality iterated, it's time to discuss how integration will be handled.

6.1 Tree

With the circuit proving an output tuple exists within a Merkle tree, we need to create the tree. The tree will be implemented in C++, with the hash function FFI'd from Rust.

The tree is a l -layer tree where each layer is of alternative widths w_a, w_b . The tree is theoretically considered to be a ∞ -layer tree, where all members of branches are zero by default. The tree root is the hash within the lowest branch with only one hash.

For the hash function $\mathcal{H}_{tree}$, parameterized by the layer alternation to $\mathcal{H}_{tree_a}, \mathcal{H}_{tree_b}$, and a list of leaves `leaves`, the tree is calculated as follows:

1. $c = a$. Hash `leaves` in w_c chunks with $\mathcal{H}_{tree_c}$ to create $\lceil \text{leaves.length}() / w_c \rceil$ members of the next layer (denoted `members`). Any incomplete chunks are padded with Z (defined later).
2. If $c = a$, $c = b$, else $c = a$. If `members.length() == 1`, the algorithm terminates. Else, hash `members` in w_c chunks with $\mathcal{H}_{tree_c}$ to create $\lceil \text{members.length}() / w_c \rceil$ members of the next layer (which become the new `members`), again padding incomplete chunks with Z .
3. Repeat the prior step.

We expose the following external API, minimizing the intricacies of the tree:

- **Grow.**
Grow incorporates a list of n leaves (themselves tuples (O, I, C)) into the tree.
- **Trim.**
Trim removes a list of the n most-recently-added leaves from the tree.

With the tree calculation described, and the API defined, we now detail performant implementations of both functions. We take extensive advantage of how the hashes are additively homomorphic ($\mathcal{H}_{tree_c}([A, 0]) + \mathcal{H}_{tree_c}([0, B]) = \mathcal{H}_{tree_c}([A, B])$).

6.1.1 Grow

For `outputs`, a set of output tuples (O, I, C) , we separate them as

$$\text{outputs}_0 = \text{outputs}[\dots w_a - (\text{tree.length()} \bmod w_a)]$$

$$\text{outputs}_1 = \text{outputs}[(\text{tree.length}() \bmod w_a) \dots]$$

Add $\mathcal{H}_{tree_c}([0, 0, 0] * (\text{tree.length}() \bmod w_a)) + \text{outputs}_0.\text{flatten}()$ to the most recently appended branch hash for the leaves. The flatten operation transforms

$$[(A, B, C), (D, E, F)]$$

to

$$[A.x, A.y, B.x, B.y, C.x, C.y, D.x, D.y, E.x, E.y, F.x, F.y]$$

(a series of tuples of points to a series of field elements).

Then, for each w_a chunk of outputs_1 , padding the last chunk with Z if incomplete, hash the flattened chunk, add B_{curve} , and append the resulting point to the next layer.

Please note this technically makes the width of the first layer $6 * w_a$, due to considering six field elements (each one word of the hash function) as individual leaves.

For each branch hash modified from H to H' , calculate $H'.x - H.x$, the delta. With the proper alignment and chunking, hash the deltas and add the resulting hash to the proper branch hash on the next layer up. New branches are considered modified from Z to H' . Repeat until the only hash modified is the new tree's root.

6.1.2 Trim

1. For every complete branch of leaves encompassed, remove them.
2. For every branch of leaves partially modified, hash the removed leaves minus Z (properly aligned within the hash function), subtracting the result from the existing branch hash if the amount of removed leaves is less than the amount of remaining leaves. If the amount of remaining leaves is less, hash those with the expected padding of Z to re-calculate the hash entirely.
3. For every hash whose children were modified, perform the above remove/delta-hash procedure until the new tree's root is updated.

6.1.3 Z

Z refers to the zero element present on every branch which is only partially full. Ideally, Z is 0, yet Z must be an x -coordinate which does not lie on the curve. For Helios and Selene, no point with 0 as its x -coordinate lies on the curve. For Wei25519, a curve birational to Ed25519 we use within the tree, 0 is not within the prime-order curve. Since following paragraphs coerce non-prime-order elements into prime-order elements, and since the circuit internally only produces prime-order elements, non-prime-order elements are effectively off curve. This lets us use 0 for Z for Wei25519 as well.

6.1.4 Initialization

When an instance of monerod boots with the tree code, it will immediately start indexing every block since genesis. Cryptonote outputs will be accumulated as the key, the linking tag generator, and a Pedersen commitment for the amount (with a randomness of zero). RingCT outputs will be accumulated as the key, the linking tag generator, and their Pedersen commitment.

In order to ensure a lack of torsion elements present, keys and commitments will be multiplied by the inverse of eight (Ed25519's cofactor) before being multiplied by eight (clearing any small-order components).

6.1.5 Normal Operation

When a block achieves a depth of 10 (the constant 10 being from the 10-block lock), all outputs within it are used to grow the tree. On a reorganization exceeding 10 blocks, trim is called for the removed outputs. On a reorganization less than 10 blocks which shortens the length of the chain, trim is called for the outputs within blocks no-longer 10-deep.

6.1.6 Timelocks

If an output has a block-based timelock, it is set to be accumulated with the outputs of that block. If an output has a time-based timelock prior to the activation of the FCMP++ hard fork, it is converted to an estimated block-based timelock and accumulated with the outputs of that block. After the FCMP++ hard fork, time-based timelocks will be rejected entirely.

Preserving time-based timelocks would require defining a linked list where upon block addition, we check if we should add the output at the head of the list (and any further outputs) to the tree. Unfortunately, such an implementation would be quite easy to perform denial-of-service attacks against. We could also define a vector, enabling efficient insertion, yet then we must define and maintain an unbounded global list which at best can be bucketed (though we'd need to remove from the head of the list, shifting the bucket(s)). With block-based timelocks, we still have the unbounded list commentary, yet have efficient topics (the block number) and no partial list additions. Accordingly, one is not worth the headache, and one has a tolerable headache.

6.2 Input Modifications

To minimize modifications to inputs, FCMP++ inputs simply set the amount of key offsets 0.

6.3 Output Modifications

New outputs' keys are transmitted multiplied by the inverse of eight. When a wallet is scanning an output, it first multiplies the key by eight. Before addition to the tree, the key is multiplied by eight. This ensures a lack of torsion.

Output commitments are already passed to the Bulletproof+ verifier multiplied by the inverse of eight. We either have commitments broadcast as such in the first place, or also pass this representation to the tree.

6.4 Modifications to RingCT Base

We define a new RingCT type, FCMPPPlusPlus, and with it a new field, ReferenceBlock. ReferenceBlock is the 32-byte hash of the Monero block which was most recently accumulated into the tree (and therefore must be at least 10 blocks old).

6.5 Modifications to RingCT Prunable

We extend prunable with a byte-buffer which is of fixed-length with regards to the amount of outputs. This byte buffer is entirely left to Rust to interpret.

This byte buffer contains one Generalized Schnorr Protocol per input (as they don't sufficiently benefit from being merged into a single proof).

It also contains one Generalized Bulletproof, containing the circuit evaluations for each input. This Generalized Bulletproof must be padded to the nearest power of two, which does computation to be wasted (one Generalized Bulletproof for 9 inputs has the verification cost of 16 inputs). The solution to this is to introduce a fee clawback, as we have for the Bulletproof(+) used to prove the outputs' amount commitments are within range, and an input limit, as we have an output limit.

6.6 Modifications to Transaction Verification

ReferenceBlock is confirmed to be on the best chain and 10-blocks deep. The tree root after applying that block is fetched. The Rust code is called with the tree root, the linking tags, the pseudo-outputs, the byte buffer from RingCT prunable, and the hash of the transaction as used for signing.

As of the most recent hard fork (Bulletproofs+), the only fields from prunable which are hashed when signing are the Bulletproofs. Moving forward, no fields from prunable are hashed for the purposes of signing. RingCT base is modified to exclude ReferenceBlock when being serialized for the hash for signing. While this would imply ReferenceBlock should be placed within prunable, this field cannot be pruned (as it's necessary to evaluate if the transaction should be dropped upon deep reorganization). The lack of hashing it, and malleability regarding the tree used, is explicitly allowed in order to support transaction chaining.

6.7 Modifications to Addresses

Addresses do not need modifications. All existing addresses will continue to work as-is. Wallets will work without modifying their keys at all.

6.7.1 Outgoing View Keys

In order to take advantage of the outgoing view key functionality, changes to the wallet's internals must be made. This is completely optional to the wallet and can be done at any time without impacting privacy. If it required generating a new address type, that would segment users (impacting privacy), hence the explicit statement there is no impact to privacy.

Wallets would not only generate the new private key, y , they would also design and develop a format for sharing x , the outgoing view key, and update software to calculate linking tags when scanning outputs if they had the incoming view key and x . They would also specify their address with the public spend key $x \cdot G + y \cdot T$, instead of $x \cdot G$. This change is indistinguishable to a recipient of an address and indistinguishable to an adversary with a discrete log oracle.

6.7.2 Forward Secrecy

Forward secrecy requires a T term be present within output keys. Practically, this can be achieved by doing the necessary modifications for outgoing view keys (which add such a term and the surrounding address/wallet modifications). No further modifications would be required.

6.8 Modifications to the RPC

The RPC is extended with a route to return the path for an output (by index) within a specified tree. The existing decoy routes are maintained. A new distribution is added comprehensive to both the Cryptonote outputs and the RingCT outputs.

6.9 Modifications to Wallets

Wallets fetch the unified distribution, yet do not fetch the decoy information. Instead, they fetch the path of each selected decoy (and the actual output).

Alternatively, wallets may locally build the tree while scanning the blockchain. This removes the need to request paths for any outputs, as wallets may now locally request the path for their output.

Instead of calling CLSAG's prove with the decoy information, wallets would call the FCMP++ prove with the path of the output being spent. No other changes to wallets are required for full-chain membership proofs (solely for additional, wallet-optional functionality, for which the changes can be done at any time).

6.10 Multisignature Wallets

Existing Monero multisignature wallets are still usable. We maintain the DKG process, which effectively creates a n-of-n multisig. We also do not touch wallet scanning/state management, nor utilities (linking proofs).

Multi-party transaction construction isn't modified. The actual signing code is modified. The existing CLSAG code is removed, replaced with either a C++ implementation of multisig OR calls to Rust multisignature code. The latter would involve passing the multisignature wallet key data from C++, reformatting into a structure the Rust code can work with, and obtaining the preprocesses/signature shares from Rust. The communication and incorporation of those into the signed transaction would occur as prior handled.

For a write-up on doing this with CLSAG and the existing monero-wallet crate, please refer to <https://gist.github.com/kayabaNerve/3b2c648c623bc4ce4ca288725428ea76>.